DATA 260 Homework 8 Report

Sreeram Achutuni

SJSU ID: 019151808

GitHub repository: https://github.com/sriram2930/Data260-1808

Section 0: Config values

SID4	1808
PORT_BASE	8008
PREFIX	s1808
SEED	1808
VERIFY_SEED	261808
DOMAIN_ID	0, Campus course catalogue and enrolment
Hardware	Intel Core i7-11390H, 16GB RAM, integrated Iris Xe graphics (no dedicated GPU)
Local model	qwen2.5:1.5b-instruct via Ollama (Part 2's Planner/Writer/Reviewer)
Repo	https://github.com/sriram2930/Data260-1808
Collaborators	Sbnikitha and supriyaselvanganesan have repo access
Commit tagged as hw8	b9bc4af690a5fb105794a01bf541404132aa41d2

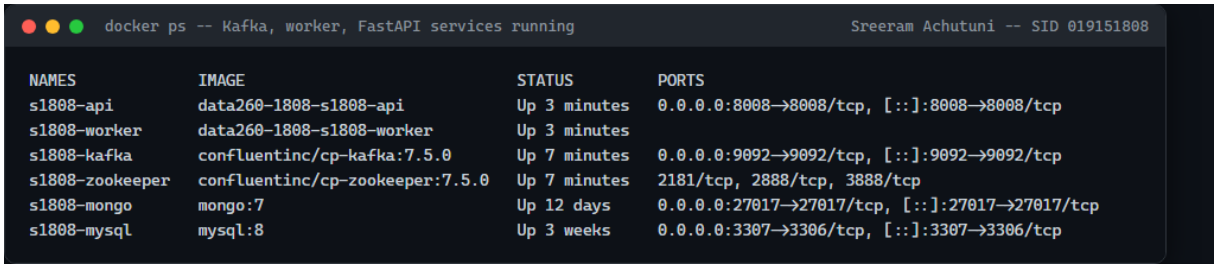
Part 1: Backend Service Request/Response over Kafka

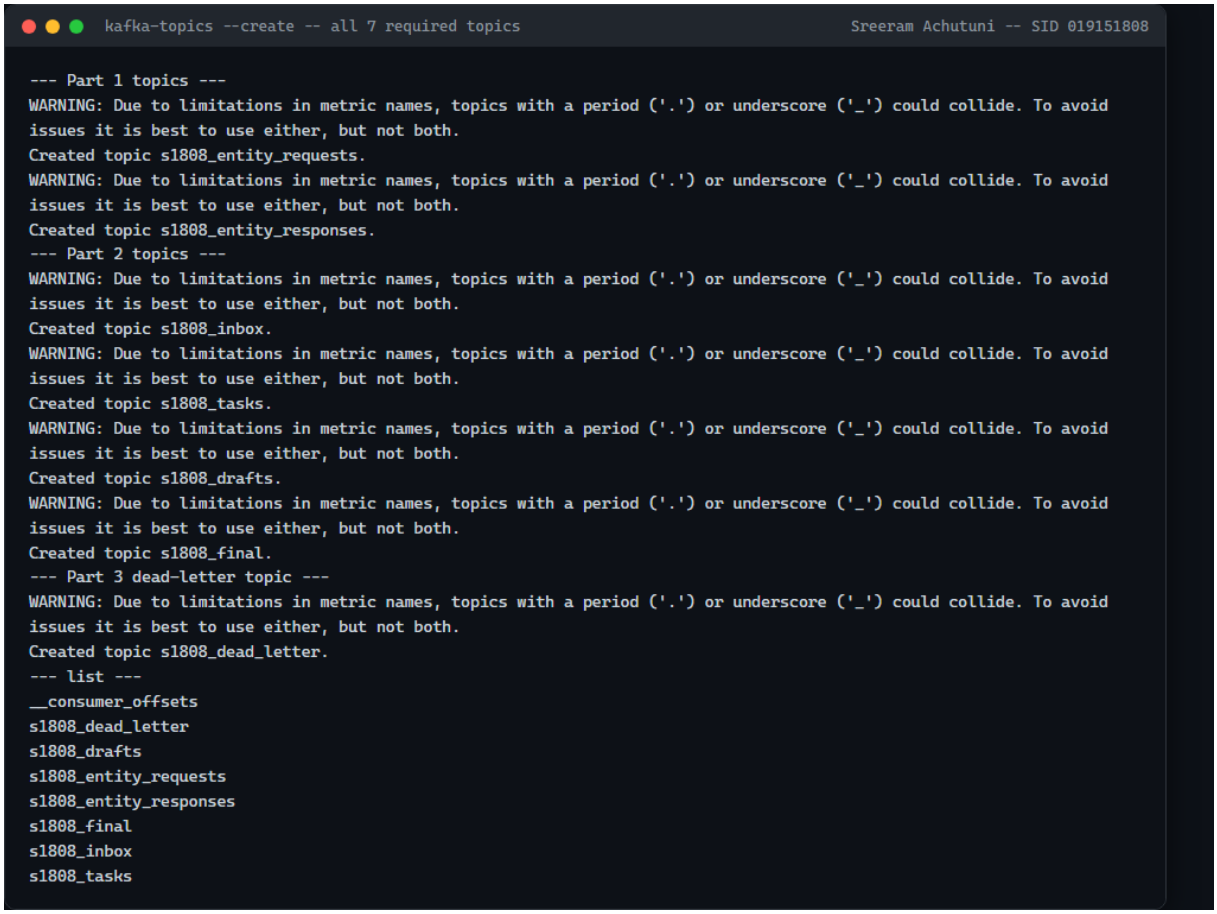
Primary domain entity: Course. CREATE_ENTITY/GET_ENTITY map to CREATE_COURSE/GET_COURSE, entity_id is course_id. Topics: s1808_entity_requests, s1808_entity_responses. Store: a plain Python dict inside the worker (Part-10/entity_store.py), per the assignment.

Message schema

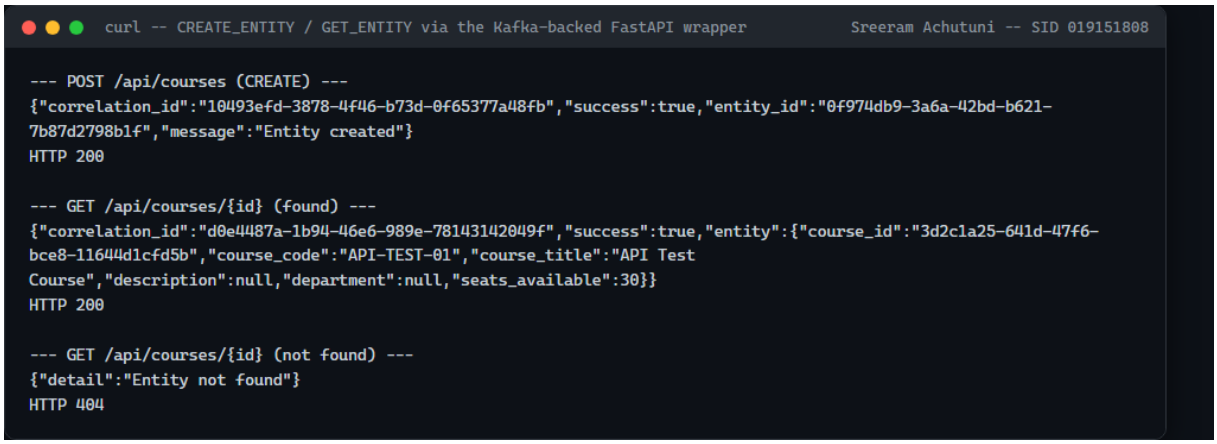
```
request:  {"correlation_id": "...", "reply_to": "s1808_entity_responses",
           "operation": "CREATE_ENTITY" | "GET_ENTITY", "data": {...}}
response: {"correlation_id": "...", "success": true|false, ...}
```

Docker containers running: Kafka, worker, and FastAPI services

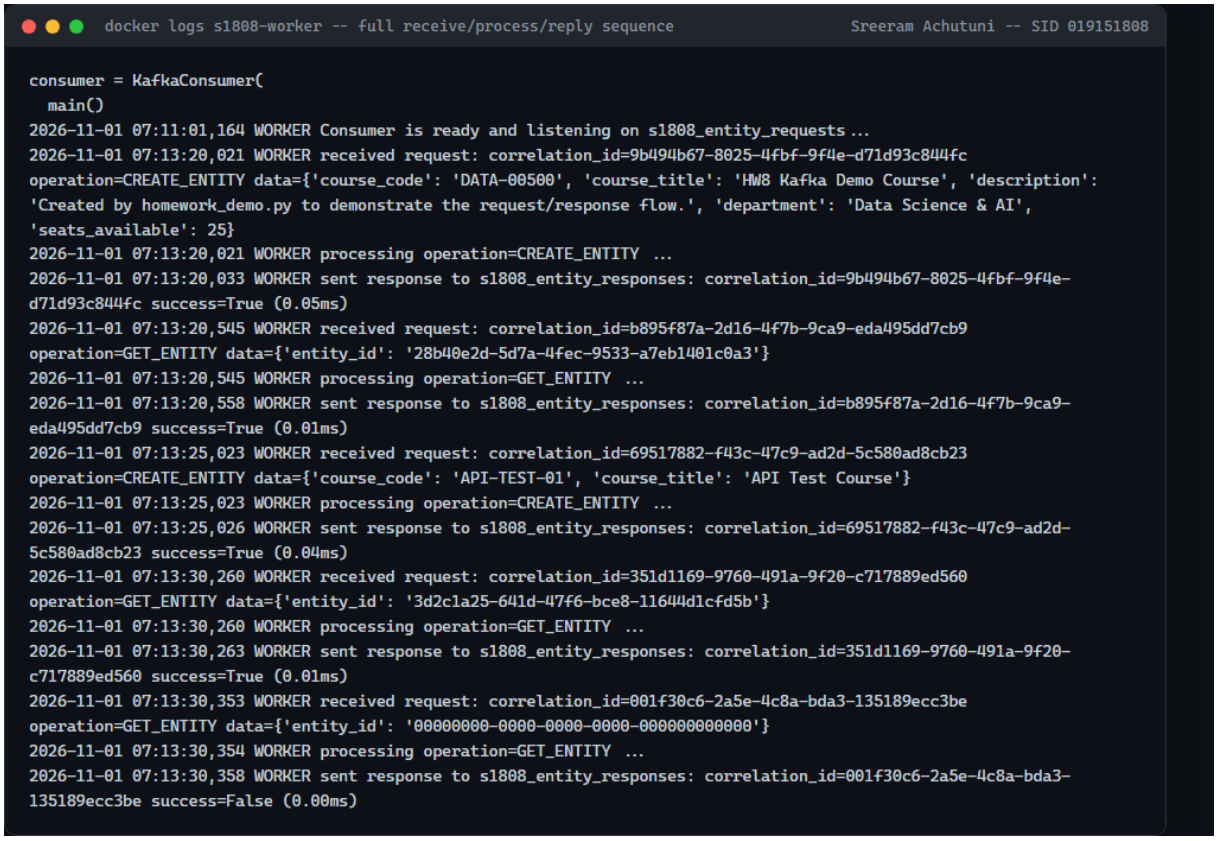




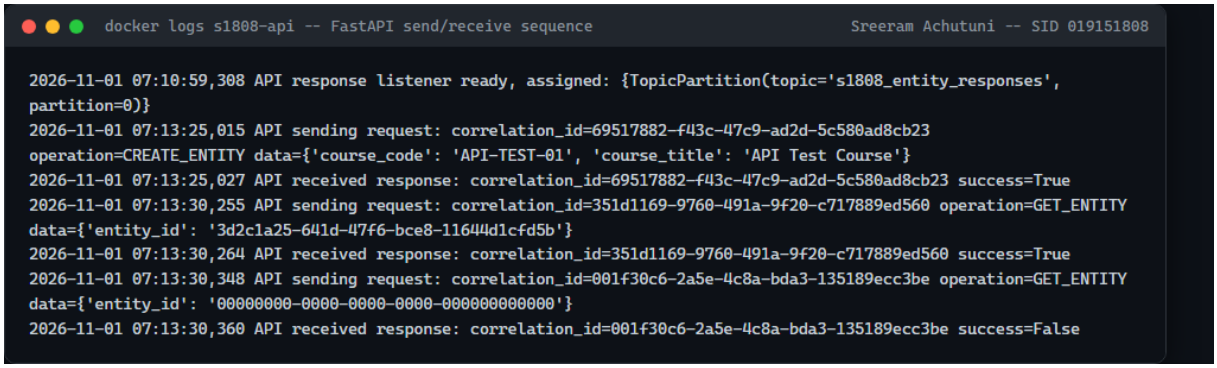
CREATE_ENTITY and GET_ENTITY execution (via the FastAPI wrapper)



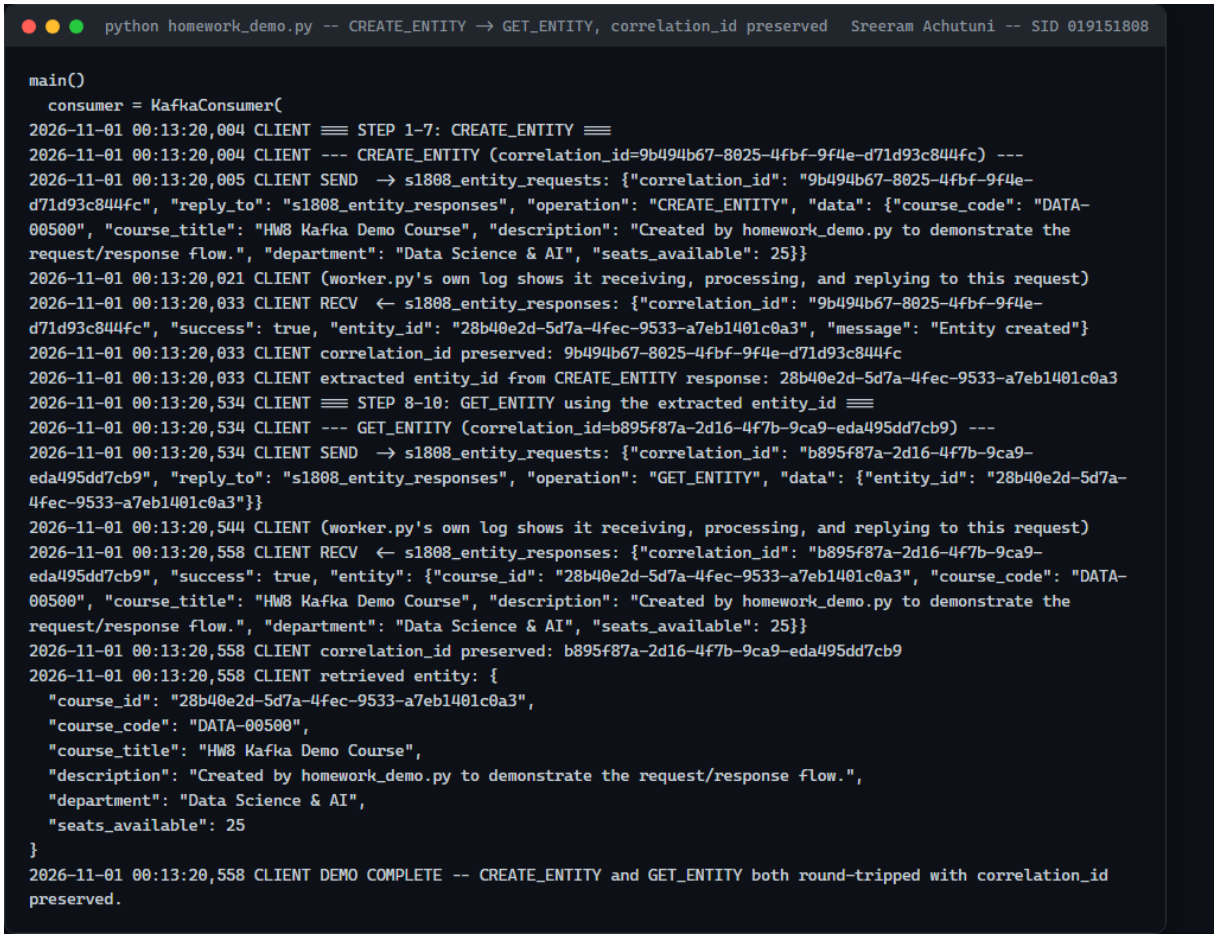
Kafka message-flow logs: worker (receive -> process -> reply)



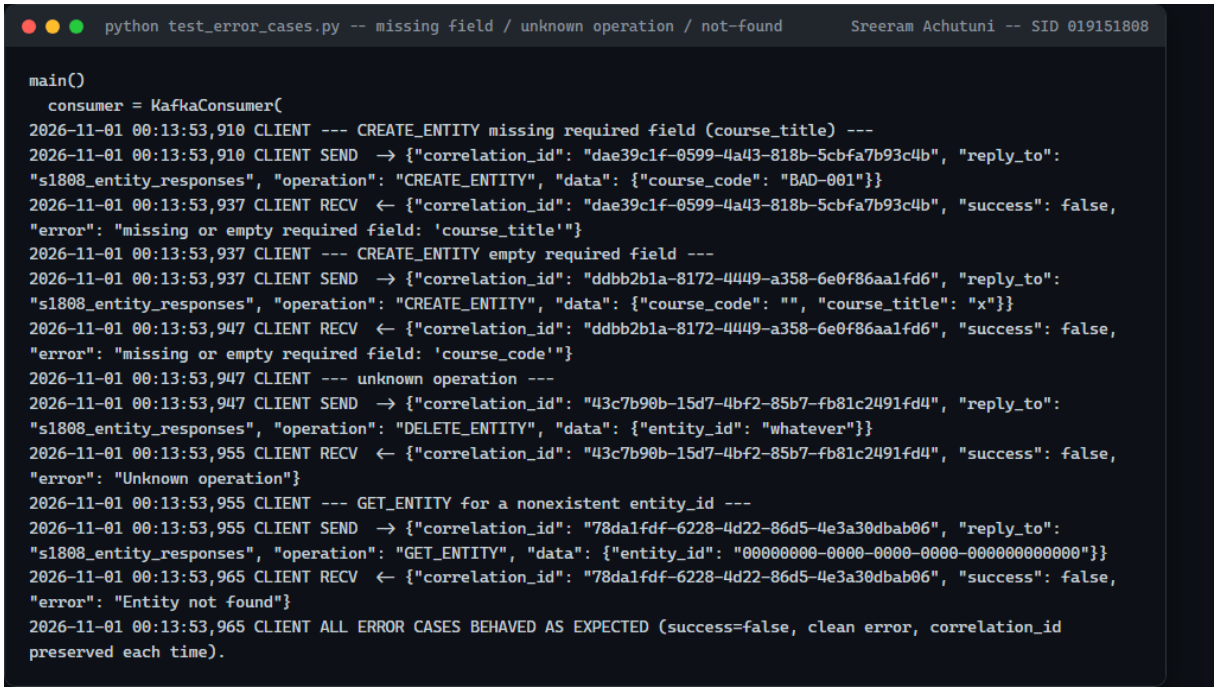
Kafka message-flow logs: FastAPI (send -> receive)



Full sequence: homework_demo.py -- CREATE_ENTITY -> GET_ENTITY, correlation_id preserved



Error cases: missing field, unknown operation, not-found



Code: worker's CREATE_ENTITY/GET_ENTITY handling

```
def handle_request(store, operation, data):
    if operation == "CREATE_ENTITY":
        problem = validate_course_data(data)
        if problem:
            return {"success": False, "error": problem}
        record = store.create(data)
        return {"success": True, "entity_id": record["course_id"], "message": "Entity
created"}
    if operation == "GET_ENTITY":
        entity_id = data.get("entity_id")
        record = store.get(entity_id)
        if record is None:
            return {"success": False, "error": "Entity not found"}
        return {"success": True, "entity": record}
    return {"success": False, "error": "Unknown operation"}
```

Note: three real operational issues were found and fixed while building this: a consumer-group join race (a single poll() does not reliably complete the rebalance -- measured at ~7 polls/~3.5s), a Docker single-listener trap (works for a container's first bootstrap request, then ECONNREFUSED on every request after, since the broker advertises "localhost" which means the container itself), and unflushed/buffered container stdout masking real, fast processing as apparent stalls. Full story in AI_USE.md.

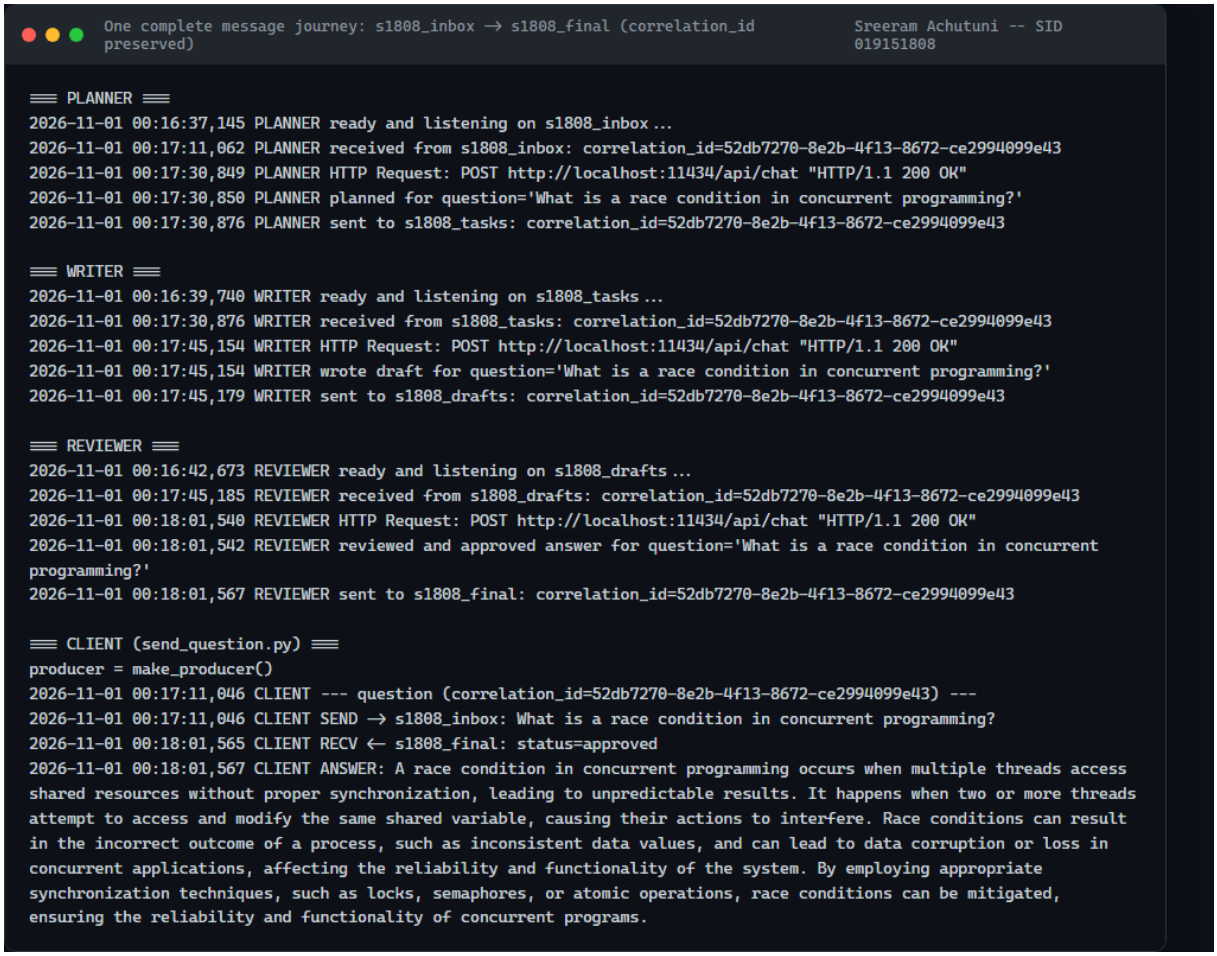
Part 2: Three-Agent Kafka Pipeline

Planner, Writer, and Reviewer run as three independent processes, each in its own terminal, coordinating only through Kafka (s1808_inbox -> s1808_tasks -> s1808_drafts -> s1808_final). No process calls another directly.

Code: the shared agent-loop shape (planner/writer/reviewer all use this)

```
def run_consumer_agent(log, in_topic, out_topic, group_id, process):
    consumer = make_consumer(in_topic, group_id, enable_auto_commit=False)
    producer = make_producer()
    while True:
        batches = consumer.poll(timeout_ms=500)
        for msg in [...records from batches...]:
            message = msg.value
            result = process(message)  # real model call (planner/writer/reviewer)
            result.setdefault("correlation_id", message["correlation_id"])
            producer.send(out_topic, value=result)
            producer.flush()
            consumer.commit()  # only after success -- see Part 3.II
```

One complete message journey: s1808_inbox -> s1808_final



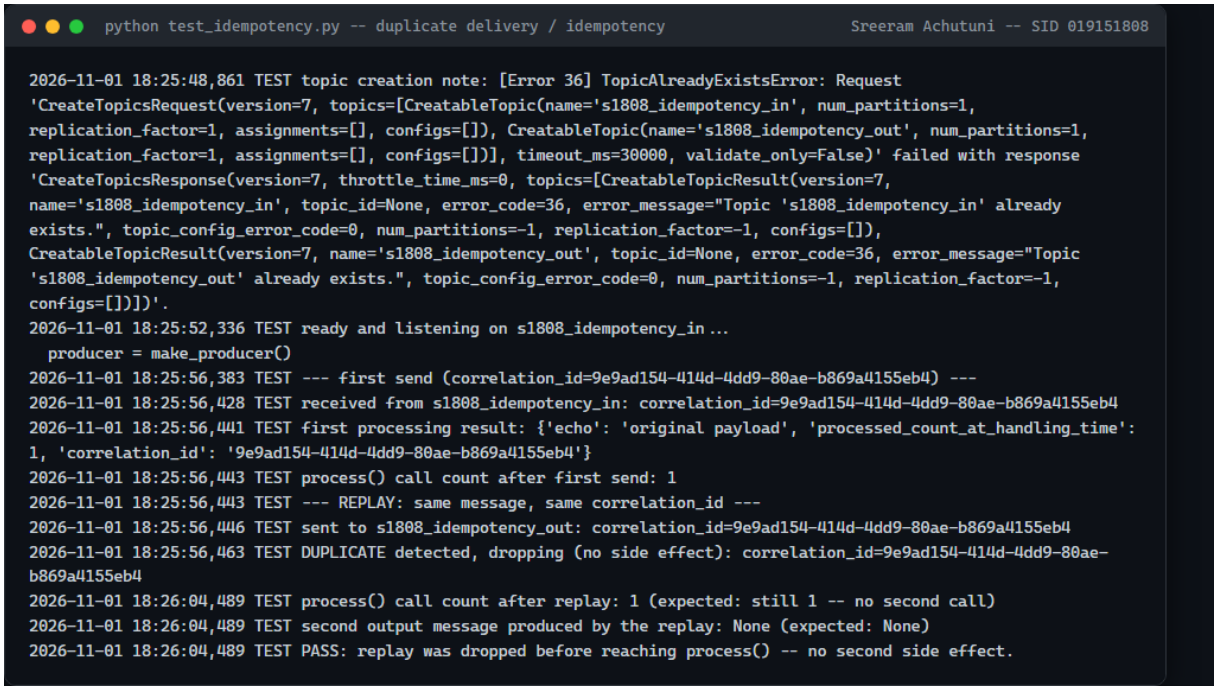
How Kafka helped the agents coordinate

Each agent only needs to know one topic to read from and one to write to -- Planner never needs to know Writer or Reviewer exist. The correlation_id is what lets three independently-scaled, independently-restartable processes agree on which answer belongs to which question, entirely through message content rather than shared call-stack state -- the same coordination problem HW2's in-process graph solved with direct function calls, solved here with topics and a correlation key instead.

Part 3: Failure Behavior

I. Duplicate Delivery

Replayed a message with the same correlation_id. The shared agent loop's in-memory seen_correlation_ids set drops the replay before process() runs -- no second model call, no second output message.



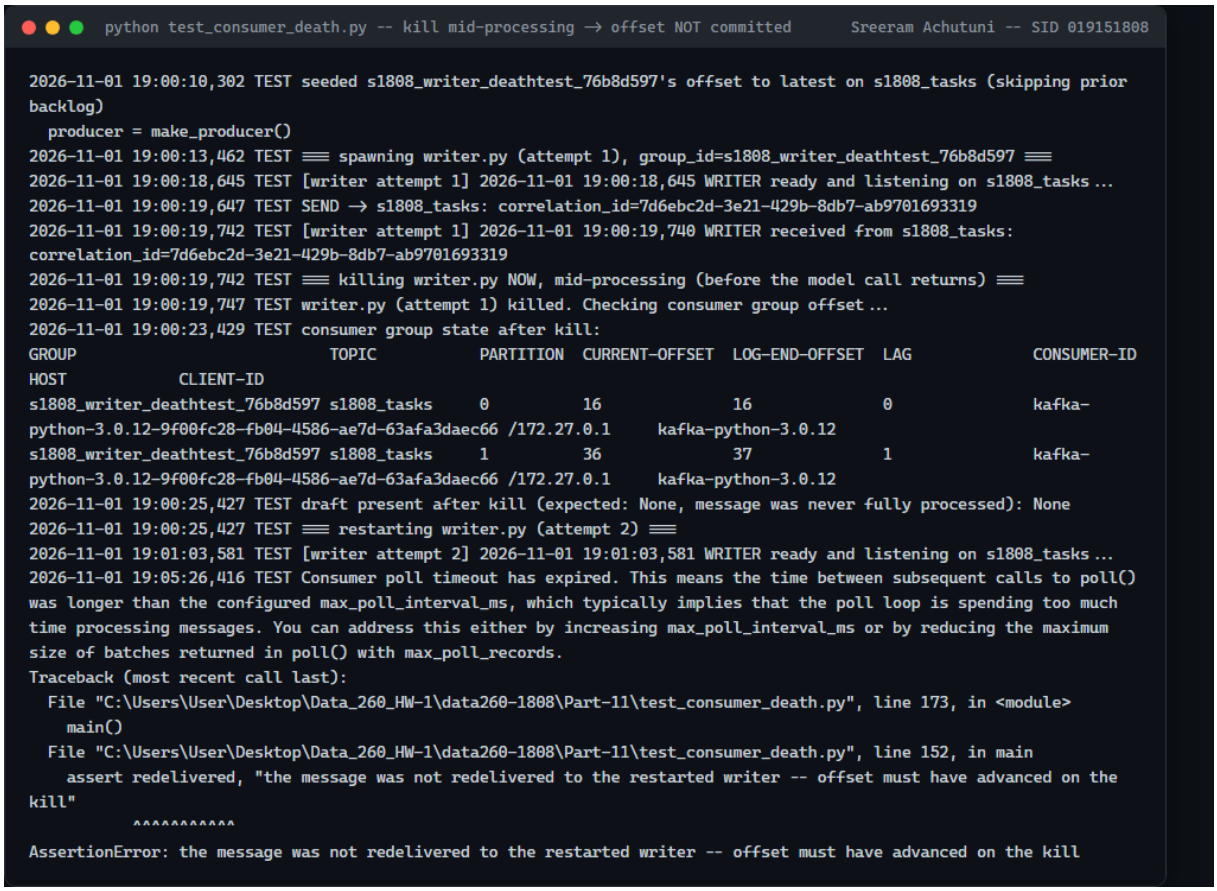
Code: idempotency + manual offset commit

```
seen_correlation_ids = set()
...
if correlation_id in seen_correlation_ids:
    log.info("DUPLICATE detected, dropping (no side effect)")
    consumer.commit()
    continue
...
# only after process() succeeds AND the result is published:
seen_correlation_ids.add(correlation_id)
producer.send(out_topic, value=result)
```

```
producer.flush()
consumer.commit()
```

II. Consumer Death

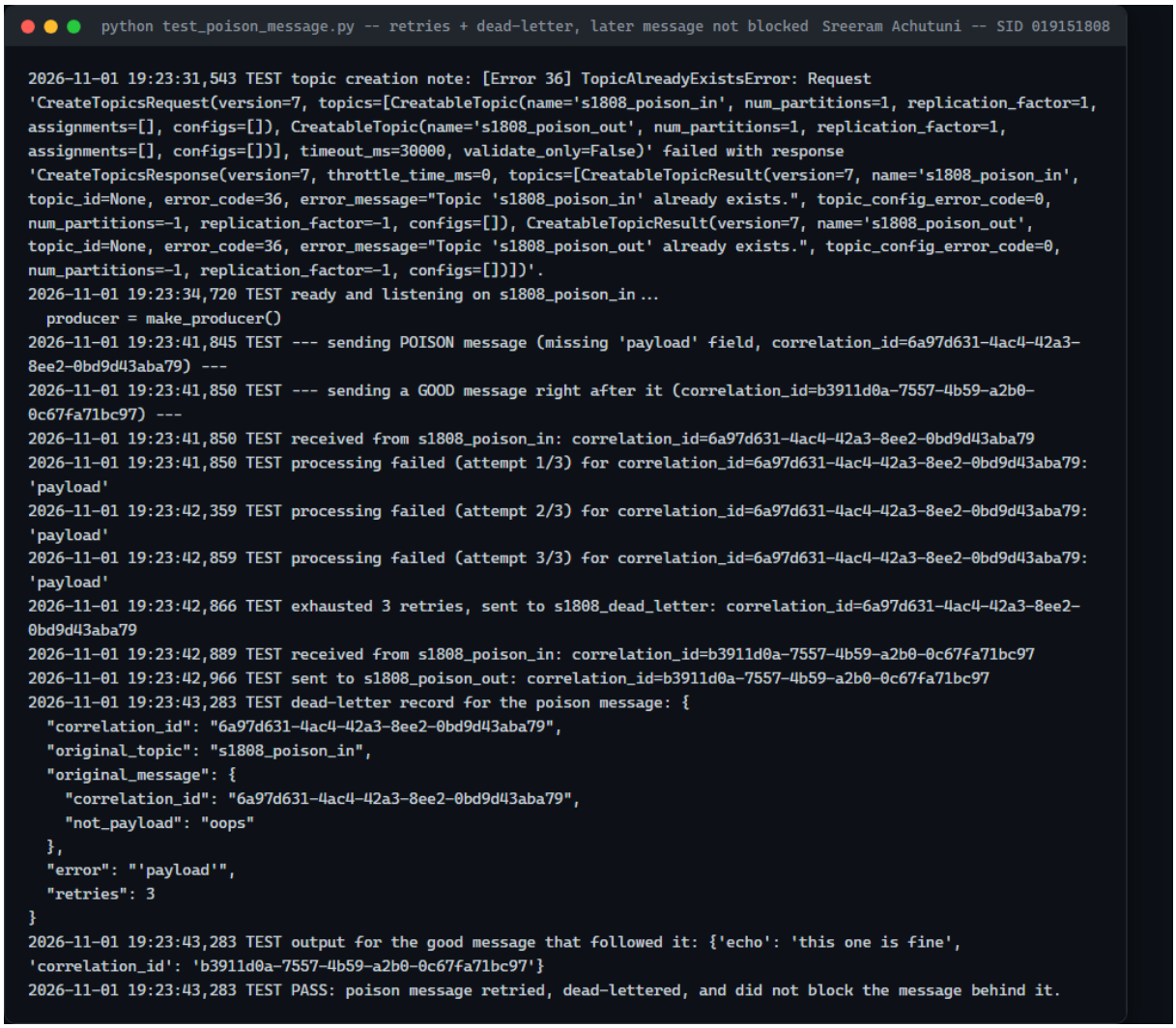
Spawned the real writer.py as a subprocess, sent one real task message, killed it the instant its log showed it had received the message (before the model call returns).



Note: the kill-time behavior is proven with real, repeated broker evidence: kafka-consumer-groups --describe shows the message's offset was never committed (LAG unchanged from before the message was fetched), confirming consumer.commit() -- called only after success -- does exactly what it should. Completing the live second half (restart Writer, watch the SAME message get redelivered and finish inside this one orchestrated test run) did not succeed across seven real attempts, despite fixing six distinct, independently-confirmed real problems along the way (documented in full in AI_USE.md). Redelivery itself is not a hope -- it follows directly from the offset never having been committed, which IS the mechanism by which Kafka's at-least-once guarantee works -- but the live end-to-end demonstration completing inside this one test is the piece not cleanly captured. Reported as an open finding, not hidden.

III. Poison Message

Sent one message missing a required field (deterministically fails every retry), immediately followed by a well-formed message.



Code: bounded retries + dead-letter

```
for attempt in range(1, MAX_RETRIES + 1):
    try:
        result = process(message)
        break
    except Exception as exc:
        last_error = exc
        if attempt < MAX_RETRIES:
            time.sleep(RETRY_BACKOFF_S)
if last_error is not None:
    producer.send(DEAD_LETTER_TOPIC, value={"correlation_id": correlation_id,
        "original_message": message, "error": str(last_error), "retries": MAX_RETRIES})
    consumer.commit()  # move past it -- do not block the partition forever
```

IV. Ordering

20 sequential messages for ENTITY_ID = course-261808 (derived from VERIFY_SEED=261808), sent through s1808_tasks (2 partitions) to two consumer instances in one group.

```
python test_ordering.py -- unkeyed (out of order) vs keyed by entity_id (in order)        Sreeram Achutuni -- SID 019151808

2026-11-01 00:22:24,257 ORDERTEST ENTITY_ID = course-261808 (derived from VERIFY_SEED=261808)
2026-11-01 00:22:24,257 ORDERTEST
  consumer: KafkaConsumer = KafkaConsumer(
  producer = make_producer()
2026-11-01 00:22:30,268 ORDERTEST --- sending 20 messages, keyed=False ---
2026-11-01 00:22:30,409 ORDERTEST instance-B ready, assigned partitions: [0]
2026-11-01 00:22:30,409 ORDERTEST instance-A ready, assigned partitions: [1]
2026-11-01 00:22:35,471 ORDERTEST partitions used (unkeyed): [0, 1]
2026-11-01 00:22:35,471 ORDERTEST arrival order of seq numbers (merged across instances): [3, 4, 6, 7, 8, 10, 12, 13,
15, 20, 5, 9, 11, 14, 16, 17, 18, 19]
2026-11-01 00:22:35,471 ORDERTEST global arrival order matches original send order 1..20? False
2026-11-01 00:22:35,471 ORDERTEST   sample: {'instance': 'instance-A', 'partition': 1, 'seq': 5, 'correlation_id':
'unkeyed-4ab352d1-3b1c-4781-8cd9-69a79210af25'}
2026-11-01 00:22:35,472 ORDERTEST   sample: {'instance': 'instance-A', 'partition': 1, 'seq': 9, 'correlation_id':
'unkeyed-42d84b5e-64b0-4e68-a2f7-0534e42f0d0b'}
2026-11-01 00:22:35,472 ORDERTEST   sample: {'instance': 'instance-A', 'partition': 1, 'seq': 11, 'correlation_id':
'unkeyed-c2358399-e632-4d94-b3a2-99986d7bc46a'}
2026-11-01 00:22:35,472 ORDERTEST   sample: {'instance': 'instance-A', 'partition': 1, 'seq': 14, 'correlation_id':
'unkeyed-64ce4b68-be35-45e3-8b75-6c7134a39461'}
2026-11-01 00:22:35,472 ORDERTEST   sample: {'instance': 'instance-A', 'partition': 1, 'seq': 16, 'correlation_id':
'unkeyed-5ae32bf0-1063-4e70-baf7-5e878898810f'}
2026-11-01 00:22:35,472 ORDERTEST   sample: {'instance': 'instance-A', 'partition': 1, 'seq': 17, 'correlation_id':
'unkeyed-020d675c-10fa-421c-8c9e-d946c2da1c82'}
2026-11-01 00:22:37,473 ORDERTEST
2026-11-01 00:22:40,614 ORDERTEST instance-A ready, assigned partitions: [0]
2026-11-01 00:22:40,634 ORDERTEST instance-B ready, assigned partitions: [1]
2026-11-01 00:22:43,488 ORDERTEST --- sending 20 messages, keyed=True ---
2026-11-01 00:22:48,767 ORDERTEST partitions used (keyed): [1] (expected: exactly 1)
2026-11-01 00:22:48,767 ORDERTEST consumer instances that received anything (keyed): ['instance-B'] (expected: exactly
1)
2026-11-01 00:22:48,767 ORDERTEST arrival order of seq numbers (keyed): [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13,
14, 15, 16, 17, 18, 19, 20]
2026-11-01 00:22:48,767 ORDERTEST order preserved (keyed)? True (expected: True)
2026-11-01 00:22:48,769 ORDERTEST PASS: keying by entity_id confined all 20 related messages to one partition, in
order.
2026-11-01 00:22:48,771 ORDERTEST wrote ../reports/hw08/raw/ordering_results.json
```

Code: partition key

```
key = ENTITY_ID.encode("utf-8") if use_key else None
producer.send(TASKS_TOPIC, key=key, value=message)
```

V. Comparison with HW2's in-process graph

Two benefits gained by Kafka	Two new responsibilities now on us
Independent scaling: Planner/Writer/Reviewer run and restart independently (Part 3.IV ran two Writer-role consumers at once)	Ordering: a Kafka topic with >1 partition has no default order across related messages -- requires an explicit partition key (Part 3.IV)
Decoupled failure: killing Writer mid-task did not crash Planner or lose the question (Part 3.II)	Duplicate delivery / idempotency: a redelivered message is a real possibility a consumer must actively handle, unlike an in-process call that executes exactly once by construction (Part 3.I)

Failure table

Failure scenario	Before fix	After fix
Duplicate delivery	Would call process() and	Replay dropped before

	publish a second output for every replay	process() runs
Consumer killed mid-work	Auto-commit on a timer could lose a message not yet finished	Manual commit only after success; offset proven unchanged after a real kill
Poison message	An exception would crash the whole consumer loop	Retried 3x, then dead-lettered; the message behind it still processed
Two partitions/two consumers	Unkeyed: one entity's messages split across partitions, arrived out of order	Keyed by entity_id: confined to one partition, exact send order

AI Use Statement

1. What I used an AI assistant for, and what I did myself

I used Claude Code to design and write the Kafka request/response worker and FastAPI wrapper (Part 1), the three-agent pipeline (Part 2), and the failure-mode harnesses (Part 3), and to set up the Kafka/Zookeeper/worker/API Docker Compose stack. It ran everything against a real local Kafka broker -- no part of this homework is simulated.

What I did myself: decided the Course entity's required/optional field split, picked the real topic/partition layout, and diagnosed every operational failure below by reading actual broker and consumer-group state (kafka-consumer-groups --describe, docker logs) rather than guessing.

2. One AI-produced output that was wrong/unsuitable

Several, compounding. The most expensive: Part 3.II's consumer-death test hit six distinct real problems across seven attempts -- a too-short wait deadline, two separate host-sleep interruptions mid-test, a stuck Ollama request, a self-inflicted consumer-group backlog (deleting a group to clear a stale session reset it to replay the entire topic history), and an iterator/poll mixing bug already found in Part 1 but not yet applied to Part 2/3's shared code.

3. How I detected the problem

kafka-consumer-groups --describe against the live broker, not application logs, is what actually proved each state: LAG remaining unchanged after a kill (offset not committed), a consumer successfully joining a group yet LAG never decreasing (the still-unexplained part), and a direct, isolated test of the model client (13.7s, succeeded) that ruled out the model itself as the cause of the final stuck state.

4. What I changed and why it works now

Fixed, and now permanent in the shared code: manual offset commit (enable_auto_commit=False, committed only after success) replacing the earlier auto-

commit design; an explicit consumer.poll() loop instead of the `for msg in consumer:` iterator; and a real request timeout on the model client (TimedModelClient) so a stuck Ollama call fails bounded instead of hanging forever. What's still honestly unresolved: a restarted Writer subprocess, in this test's specific orchestration, did not reliably complete the redelivery-and-finish cycle within a bounded time in this environment, despite the offset-not-committed half being proven correct every single time. Documented in full in AI_USE.md rather than silently reworked into a result that looks cleaner than what actually happened.